

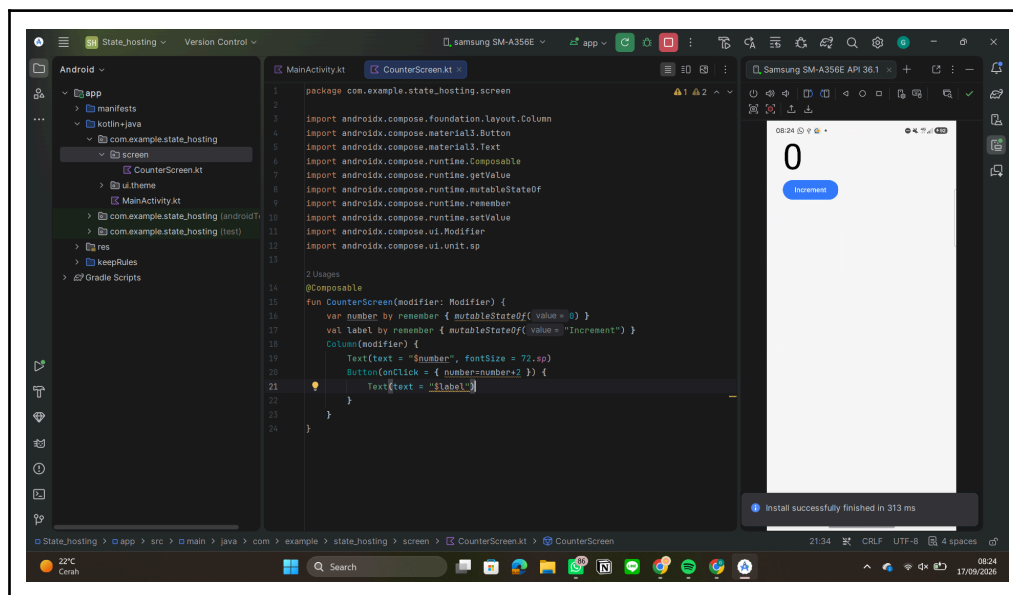
Nama : Givenia Marudur Hasiana
NIM : 245150400111051
Mata Kuliah : Pengembangan Aplikasi Mobile
Kelas : SI - F

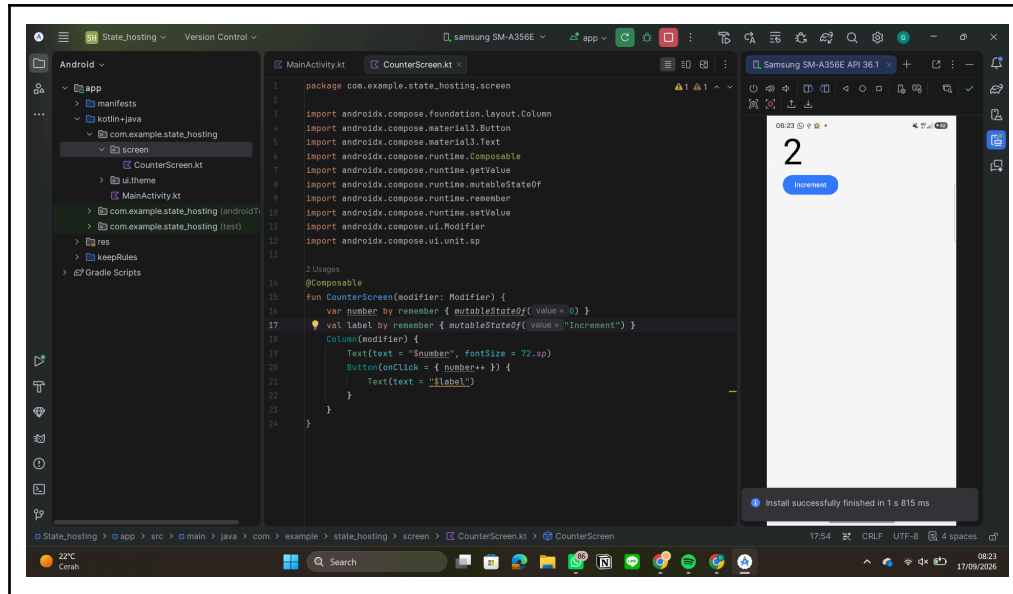
MODUL 4

STATE HOISTING, PERUBAHAN KONFIGURASI, DAN SIDE-EFFECT

1. State Hoisting

- 1.1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian klik tombol "Increment" beberapa kali, ambil satu screenshot setelah tombol "Increment" diklik beberapa kali.





a. Apa yang terjadi setiap kali tombol "Increment" diklik?

Angka yang ditampilkan pada layar bertambah 1 setiap kali tombol tersebut diklik.

b. Apa yang menyebabkan hal tersebut terjadi?

Hal ini terjadi karena variabel number dideklarasikan sebagai state menggunakan remember { mutableStateOf(0) }. Setiap kali onClick pada Button dijalankan, statement number++ mengubah nilai state tersebut, yang kemudian memicu Compose untuk melakukan recomposition pada composable Text yang menampilkan nilai number.

c. Variabel state apa yang berperan pada Composable Text yang menampilkan perubahan angka pada Text tersebut?

Variabel number.

d. Variabel state apa yang berperan untuk menampilkan label teks pada Composable Button yang bertuliskan "Increment" tersebut?

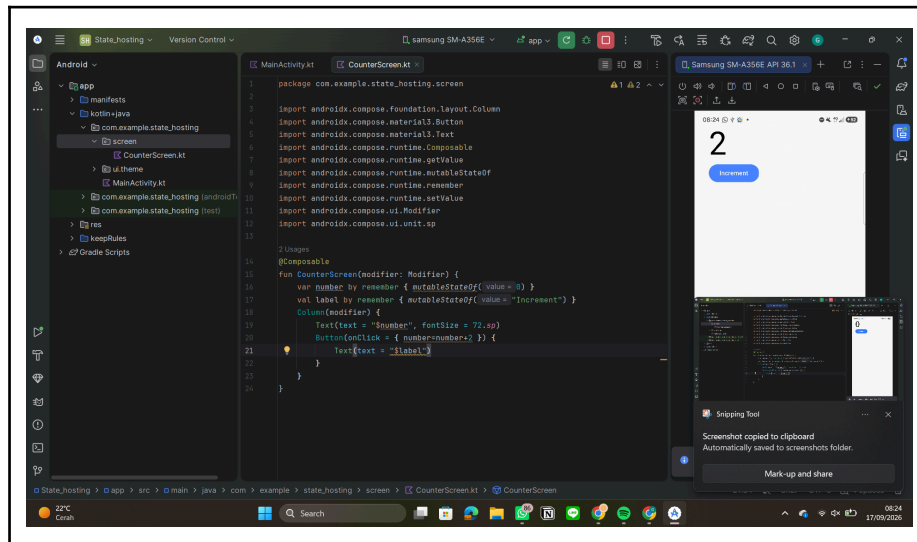
Variabel label.

1.2. Pada percobaan tahap sebelumnya, ubah kode program pada Composable CounterScreen agar setiap kali tombol "Increment" diklik, nilai yang ditampilkan menjadi increment 2 (bertambah 2 setiap klik). Ubah pula label tombol "Increment" menjadi "Increment 2".

a. Perubahan apa yang perlu dilakukan dan dilakukan di mana?

- Baris number++ pada onClick Button diubah menjadi number += 2 (atau number = number + 2).

- Nilai awal variabel label yang semula "Increment" diubah menjadi "Increment 2".
- b. Jalankan program dan ambil masing-masing satu screenshot sebelum dan sesudah tombol Increment diklik untuk memperlihatkan bahwa perubahan tersebut berjalan dengan baik.



1.3. Langkah selanjutnya adalah melakukan state dan event hoisting pada state dan event yang sebelumnya terdapat di dalam Composable CounterScreen.tate ini, jadi pas state-nya berubah, otomatis Text-nya ikut update sendiri.

Perhatikan bahwa deklarasi variabel state number dari yang sebelumnya dilakukan di dalam fungsi Composable CounterScreen, sekarang dideklarasikan di dalam fungsi onCreate() sebelum fungsi Composable CounterScreen dipanggil dari dalam fungsi onCreate().

1.4. Jalankan program, dan kemudian klik beberapa kali tombol Increment sehingga angka yang ditampilkan berubah.

- a. Apakah program berjalan dengan baik serta menampilkan hal yang sama persis seperti yang dilakukan pada tahap nomor 4 sebelumnya? Ya, secara fungsional dan tampilan program berjalan persis sama; angka tetap bertambah setiap kali tombol diklik, dan label tombol tetap tampil sesuai teks yang diberikan. Perbedaannya murni pada struktur kode, bukan pada perilaku aplikasi yang terlihat oleh pengguna.

- b. Di bagian kode program manakah saat ini proses increment dilakukan?

Proses increment (`number++`) kini dilakukan di dalam fungsi `onCreate()`, tepatnya pada lambda `onButtonClick = { number++ }` saat memanggil `CounterScreen`. Proses ini tidak lagi berada di dalam fungsi `Composable CounterScreen`.

- c. Di bagian kode program manakah saat ini state `Composable Text` dan `Composable Button "Increment"` dideklarasikan?

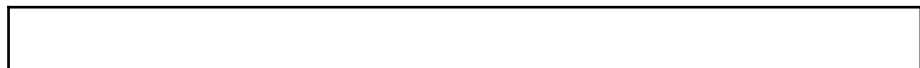
Variabel state `number` dideklarasikan di dalam fungsi `onCreate()` (menggunakan `remember { mutableStateOf(0) }`), sebelum pemanggilan `CounterScreen`. Variabel label sekarang bukan lagi state, melainkan langsung diteruskan sebagai nilai String literal `"Increment"` pada parameter label saat memanggil `CounterScreen`.

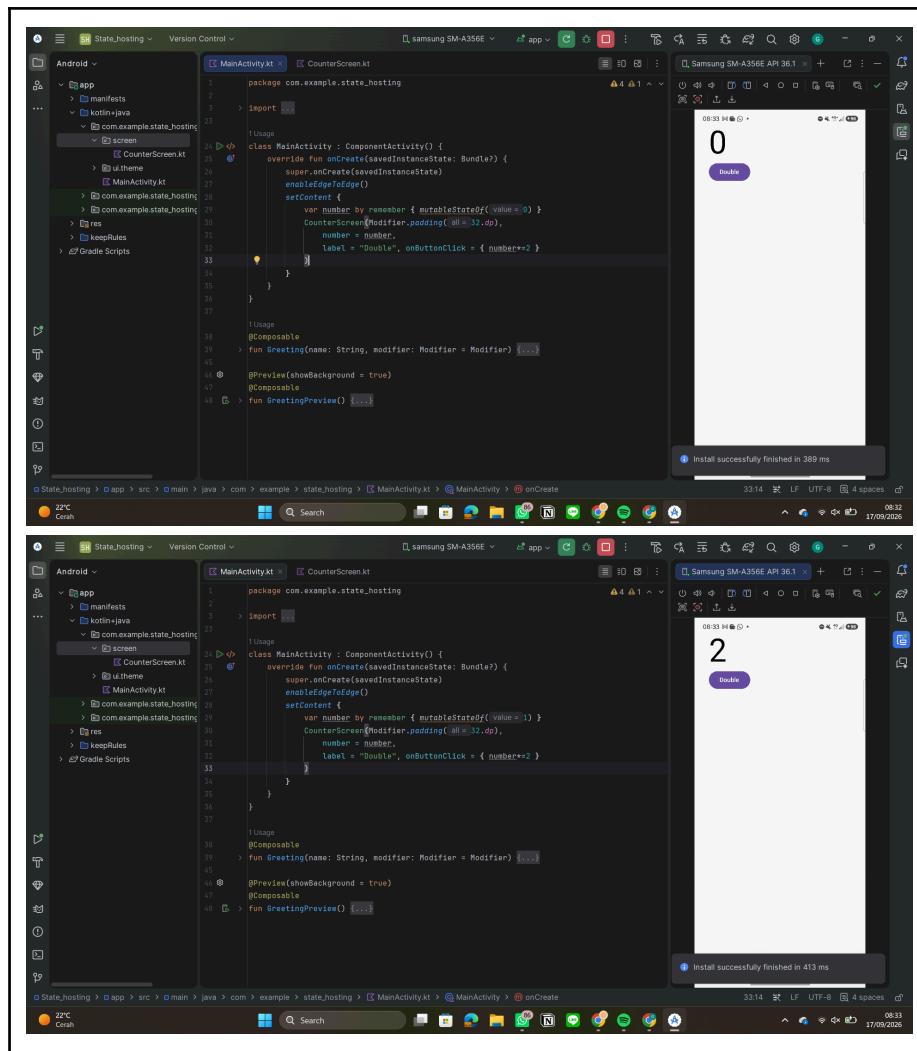
- 1.5. Diasumsikan terdapat perubahan kebutuhan fungsional aplikasi yang memaksa terjadinya perubahan logika berjalannya program pada antar muka pengguna.

- Label tombol `"Increment"` menjadi `"Double"`
- Nilai awal variabel `number` dari 0 menjadi 1.
- Ubah fungsi/lambda `onButtonClick` pada pemanggilan `Composable CounterScreen` di dalam fungsi `onCreate()` sedemikian rupa sehingga yang dilakukan dari sebelumnya: `number++` menjadi menggandakan nilai variabel `number`.
- Tuliskan bagian kode program yang mendeklarasikan state dan pemanggilan fungsi `Composable CounterScreen` di dalam fungsi `onCreate()` setelah langkah nomor 8(a), 8(b), dan 8(c) dilakukan.

- 1.6. Jalankan program,

- a. Perlihatkan dengan mengambil satu screenshot sebelum dan sesudah nilai variabel `number` digandakan.





- b. Di bagian manakah dari kode program pada percobaan tahap nomor 7 dan tahap nomor 8 dilakukan?

Seluruh perubahan (nilai awal number, teks label, dan logika onClick) dilakukan di dalam fungsi onCreate(), tepatnya pada bagian deklarasi state dan pemanggilan CounterScreen. Tidak ada perubahan yang menyentuh fungsi Composable CounterScreen itu sendiri.

- c. Apakah ada perubahan yang dilakukan dari tahap nomor 7 dan tahap nomor 8 yang dilakukan di dalam fungsi Composable CounterScreen itu sendiri? Jika ada sebutkan perubahan yang dilakukan.

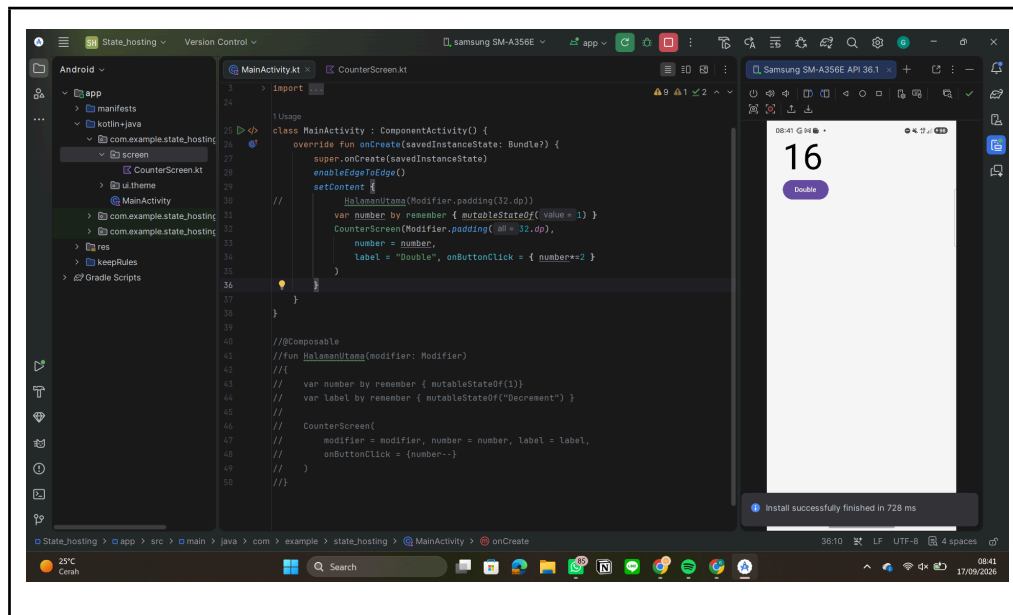
Tidak ada. Fungsi CounterScreen sama sekali tidak diubah pada tahap 7 dan 8 ini.

1.7. Apa arti dari jawaban pada tahap nomor 9 terkait dengan manfaat dari melakukan state hoisting dalam perubahan kebutuhan fungsional aplikasi yang terkait dengan antar muka pengguna?

Hal ini menunjukkan manfaat utama state hoisting: ketika terjadi perubahan kebutuhan fungsional (logika bisnis, seperti mengubah dari increment menjadi doubling, atau mengubah label tombol), perubahan tersebut cukup dilakukan pada level pemanggil (`onCreate()`) tanpa perlu menyentuh atau memodifikasi kode `Composable CounterScreen` itu sendiri. Ini membuktikan bahwa `CounterScreen` benar-benar bersifat `stateless`, `reusable`, dan independen terhadap logika bisnis spesifik — sesuai prinsip `Separation of Concern`.

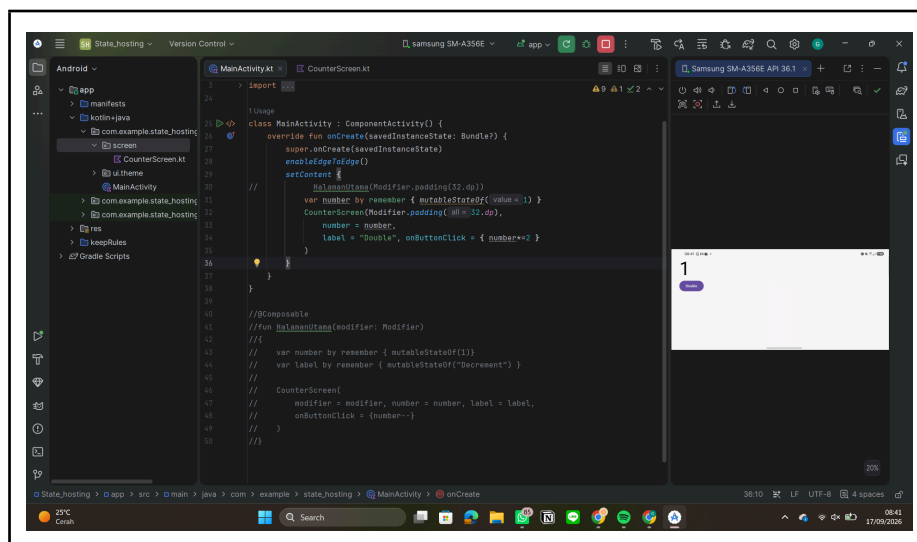
2. Perubahan Konfigurasi

2.1. Screenshot



2.2. Lakukan perubahan konfigurasi dengan memutar posisi perangkat dari posisi portrait menjadi landscape, sehingga tampilan aplikasi akan disesuaikan oleh Android.

- Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi landscape.



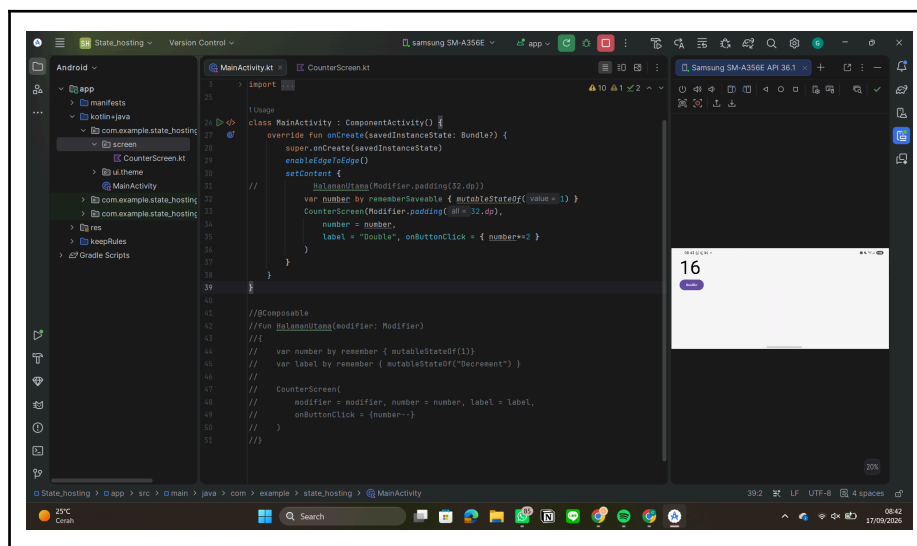
- Berapa angka yang ditampilkan? Mengapa terjadi demikian?

Angka yang ditampilkan akan kembali ke nilai awal (1), bukan melanjutkan nilai terakhir sebelum rotasi. Hal ini terjadi karena rotasi layar merupakan perubahan konfigurasi (configuration change), yang menyebabkan Activity dihancurkan dan dibuat ulang (onCreate())

dipanggil kembali) oleh sistem Android. Karena remember hanya mempertahankan state selama proses recomposition dalam siklus hidup Composable yang sama, dan tidak bertahan terhadap penghancuran-ulang Activity, maka nilai number diinisialisasi ulang dari nilai awalnya.

2.3. Lakukan kembali perubahan konfigurasi dengan memutar posisi perangkat dari posisi portrait menjadi landscape, sehingga tampilan aplikasi akan disesuaikan oleh Android.

a. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi landscape.



b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?

Angka yang ditampilkan akan tetap sama dengan nilai sebelum rotasi (tidak kembali ke nilai awal 1). Hal ini terjadi karena rememberSaveable menyimpan nilai state ke dalam objek Bundle melalui mekanisme saved instance state. Ketika Activity dihancurkan dan dibuat ulang akibat perubahan konfigurasi, sistem Android akan memulihkan nilai tersebut dari Bundle yang tersimpan, sehingga nilai number tidak hilang meskipun Activity mengalami re-creation.

c. Apa fungsi dari konstruk rememberSavable pada kode program Android yang membedakannya dari konstruk remember?

remember hanya mempertahankan nilai state selama proses recomposition dalam siklus hidup composition yang sama, dan akan hilang jika Activity dihancurkan-ulang (misalnya akibat rotasi layar). Sebaliknya, rememberSaveable mempertahankan state tidak hanya

selama recomposition, tetapi juga melewati proses penghancuran dan pembuatan ulang Activity/proses akibat perubahan konfigurasi, karena nilainya disimpan ke dalam Bundle (saved instance state) yang dipulihkan kembali oleh sistem setelah Activity dibuat ulang.

2.4. Beri contoh kasus nyata lain di mana penggunaan konstruk `rememberSavable` ini tepat digunakan dalam aplikasi Android serta sebutkan manfaat yang diperoleh dari penerapannya!

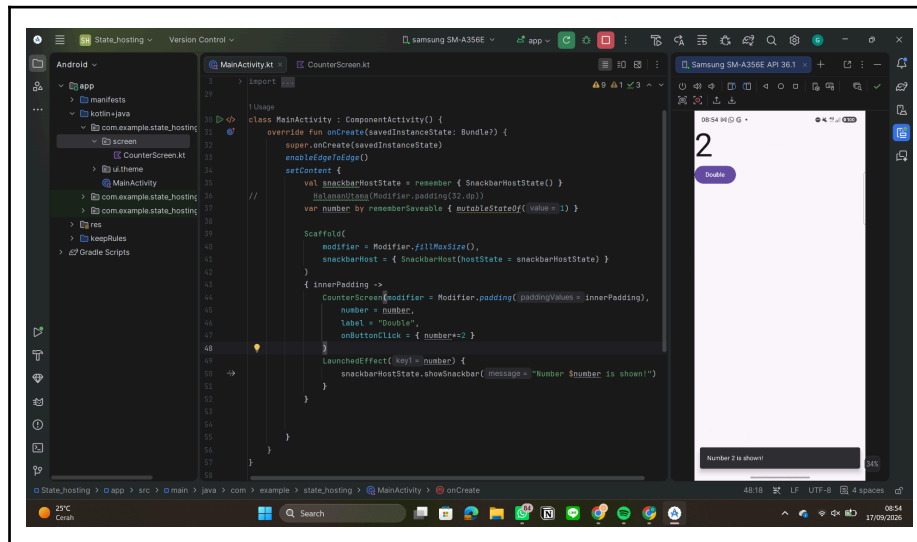
Contoh: input teks pada formulir pendaftaran (misalnya kolom nama, email, atau alamat) yang sedang diisi pengguna. Jika pengguna sedang mengisi formulir tersebut lalu secara tidak sengaja memutar orientasi perangkat, penggunaan `rememberSaveable` akan memastikan data yang telah diketik tidak hilang dan tetap ditampilkan setelah rotasi terjadi. Manfaatnya adalah meningkatkan pengalaman pengguna (UX), karena pengguna tidak perlu mengulang proses pengisian data yang sudah dilakukan hanya karena terjadi perubahan orientasi layar yang tidak disengaja.

3. LaunchedEffect

Pra-Percobaan 1

3.1. Jalankan program,

- Pada saat pertama kali program dijalankan, tanpa mengklik button "Double" apa yang ditampilkan di dalam layar?



- Mengapa pesan tersebut ditampilkan saat pertama kali aplikasi dijalankan tanpa ada interaksi apa pun dari pengguna?

Karena `LaunchedEffect(number)` akan otomatis dijalankan setiap kali `composable` yang memuatnya pertama kali memasuki komposisi (`composition`), terlepas dari ada tidaknya interaksi pengguna. Pada saat aplikasi pertama kali dijalankan, `composable` tersebut baru saja memasuki komposisi, sehingga blok kode di dalam `LaunchedEffect` — yaitu memanggil `snackbarHostState.showSnackBar(...)` — otomatis dieksekusi satu kali.

- Apa yang memicu pesan pada `SnackBar` tersebut ditampilkan? Pada bagian kode program yang mana?

Yang memicunya adalah pemanggilan fungsi `showSnackBar()` di dalam blok:

```
LaunchedEffect(number){
    snackbarHostState.showSnackBar("Number $number is shown!") }
```

3.2. Klik button "Double" satu kali,

- Apa yang terjadi?

Pesan SnackBar baru muncul kembali, kali ini menampilkan nilai number yang telah digandakan (misalnya "Number 2 is shown!").

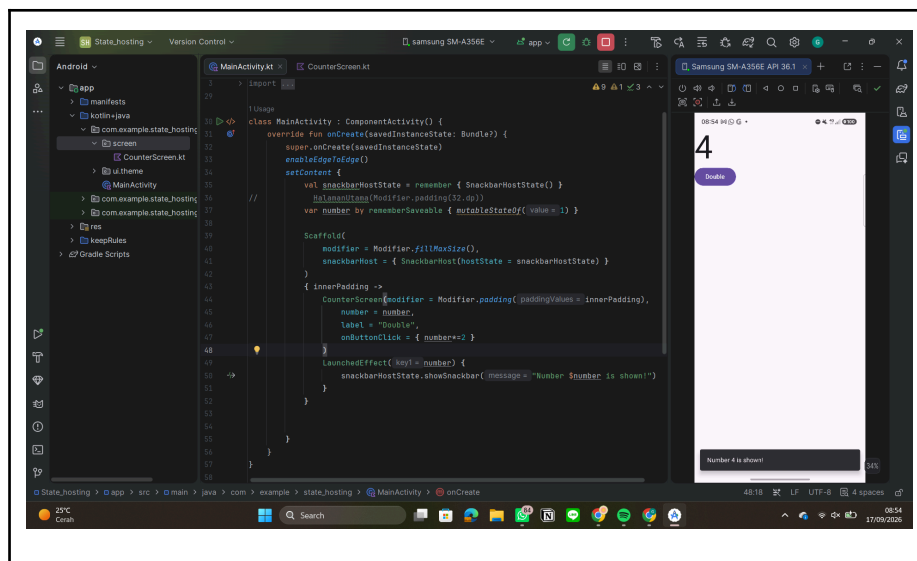
b. Mengapa hal tersebut terjadi? Jelaskan!

Karena `LaunchedEffect(number)` menggunakan number sebagai key. Setiap kali nilai key tersebut berubah, `LaunchedEffect` akan membatalkan coroutine yang sedang berjalan (jika ada) dan menjalankan ulang blok kodenya dengan nilai key yang terbaru. Karena mengklik Button "Double" mengubah nilai number, hal ini memicu `LaunchedEffect` untuk dijalankan kembali dan menampilkan SnackBar dengan nilai terbaru.

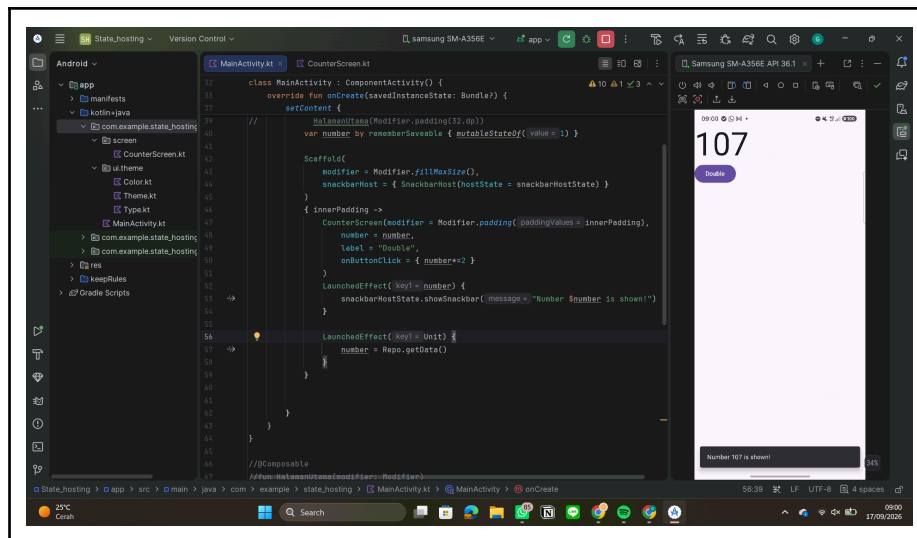
Pra-Percobaan 2

3.3. Jalankan program,

- a. Pada saat pertama kali program dijalankan, tanpa mengklik button "Double" apa yang ditampilkan di dalam layar?



- b. Tunggu selama dua detik, kemudian periksa kembali apa yang ditampilkan pada layar! Apa yang tampil?



Nilai number berubah menjadi angka acak antara 100-999 (hasil dari `Repo.getData()`), dan Snackbar baru muncul menampilkan nilai acak tersebut, misalnya "Number 248 is shown!".

- c. Apa yang memicu perubahan angka dan pesan yang ditampilkan pada Snackbar tersebut ditampilkan? Pada bagian kode program yang mana?

Yang memicu adalah blok:

```
LaunchedEffect(Unit) { number = Repo.getData() }
```

`LaunchedEffect(Unit)` dijalankan satu kali saat composable pertama kali memasuki komposisi. Di dalamnya, `Repo.getData()` (fungsi suspend yang men-delay 2 detik lalu mengembalikan angka acak) dipanggil, dan hasilnya digunakan untuk mengubah state number. Perubahan pada number inilah yang kemudian secara tidak langsung memicu `LaunchedEffect(number)` yang lain (dari Pra-Percobaan 1) untuk dijalankan ulang, karena key-nya (number) berubah — sehingga Snackbar baru ditampilkan.

- 3.4. Dari kedua percobaan terkait dengan side-effect dengan `LaunchedEffect`,
 - a. Jelaskan apa perbedaan antara `LaunchedEffect(state)` dengan `LaunchedEffect(Unit)`?

- `LaunchedEffect(state)` menggunakan sebuah variabel state (misalnya `number`) sebagai key. Setiap kali nilai state tersebut berubah, coroutine yang sedang berjalan dibatalkan dan blok kode dijalankan ulang dengan nilai terbaru.
- `LaunchedEffect(Unit)` menggunakan `Unit` (nilai konstan yang tidak pernah berubah) sebagai key. Karena key-nya tidak pernah berubah, blok kode di dalamnya hanya akan dijalankan satu kali saja, yaitu ketika composable tersebut pertama kali memasuki komposisi.

b. Kapan operasi di dalam `LaunchedEffect(state)` dengan `LaunchedEffect(Unit)` dijalankan?

- `LaunchedEffect(state)` dijalankan setiap kali composable pertama kali memasuki komposisi dan setiap kali nilai state yang menjadi key-nya berubah.
- `LaunchedEffect(Unit)` hanya dijalankan satu kali, yaitu tepat saat composable pertama kali memasuki komposisi, dan tidak akan pernah dijalankan ulang selama composable tersebut tetap berada dalam komposisi (kecuali composable tersebut sempat keluar dari komposisi lalu masuk kembali).